
PROJECT REPORT

Design and Implementation of a Database Management System for School Transport Route and Student Pickup Management System



NAME: SARDAR TRILOCHAN SINGH

ROLL NO: 25WU0102246

PROGRAM: B.Tech Artificial Intelligence and Machine Learning

COURSE NAME: DATABASE MANAGEMENT SYSTEMS

FACULTY NAME: Dr. Kiran Mayee Adavala

ACADEMIC YEAR: 2025 – 2029

WOXSEN UNIVERSITY
School of Technology

ABSTRACT

Managing school transport operations manually is a challenging task for educational institutions, particularly as the number of students, buses, drivers, and routes grows. Information related to student pickup and drop locations, route assignments, driver and bus allocation, daily pickup records, and transport fee collection is often scattered across registers and spreadsheets, leading to inconsistency, duplication, and difficulty in retrieval. This project, titled "Design and Implementation of a Database Management System for School Transport Route and Student Pickup Management System", addresses this problem by designing and implementing a centralized relational database that systematically organizes all transport-related information.

The system models nine interrelated entities — Driver, Bus, Route, Pickup Point, Drop Point, Student, Student Transport, Pickup Record, and Transport Fee — using a normalized relational schema implemented in MySQL. Primary key and foreign key constraints enforce referential integrity between drivers and buses, buses and routes, routes and pickup/drop points, and students and their transport assignments, pickup records, and fee records. A web-based administrative interface was developed using React with Vite on the frontend and Node.js with Express on the backend, communicating with the MySQL database through the mysql2 driver. The interface allows authorized users to view, search, filter, insert, and delete records across the transport management domain.

The database was implemented and populated with a complete dataset covering drivers, buses, routes, pickup and drop points, students, transport assignments, pickup records, and fee records, and a series of SQL queries were executed to validate relationships such as route-wise student distribution, pending fee analysis, and bus capacity utilization. Functional testing, including a live create-and-delete demonstration on a sample student record, was carried out to confirm the correctness of the CRUD operations and the integrity of the underlying schema. The project demonstrates the practical application of core Database Management Systems concepts — entity-relationship modelling, normalization, SQL querying, and full-stack database integration — in building a usable, centralized solution for school transport administration.

TABLE OF CONTENTS

ABSTRACT.....	2
TABLE OF CONTENTS.....	3
1. INTRODUCTION	5
2. PROBLEM STATEMENT	6
3. OBJECTIVES AND SCOPE	7
3.1 Objectives	7
3.2 Scope.....	7
4. EXISTING SYSTEM AND PROPOSED SYSTEM	8
5. SOFTWARE AND HARDWARE REQUIREMENTS	9
5.1 Software Requirements.....	9
5.2 Hardware Requirements.....	9
6. SYSTEM ARCHITECTURE	10
6.1 Frontend Layer (React + Vite).....	10
6.2 Backend Layer (Node.js + Express)	10
6.3 Database Layer (MySQL).....	10
7. DATABASE DESIGN.....	11
8. ER DIAGRAM	12
9. RELATIONAL SCHEMA AND NORMALIZATION	13
9.1 Relational Schema Overview.....	13
9.2 Normalization	13
10. DATA DICTIONARY.....	14
driver	14
bus	14
route	14
pickup_point	14
drop_point.....	15
student.....	15
student_transport.....	15
pickup_record	15
transport_fee	16
11. DATABASE IMPLEMENTATION	16
11.1 Database Creation	16
11.2 Representative Table Creation.....	16
11.3 Representative Alteration.....	16
12. DDL AND DML OPERATIONS	17

12.1 INSERT.....	17
12.2 UPDATE.....	17
12.3 DELETE	17
12.4 SELECT.....	17
13. SQL QUERIES AND RESULTS	17
Query 1: Students Under a Specific Driver	17
Query 2: Route-wise Students	18
Query 3: Complete Transport Details	18
Query 4: Pending Transport Fees	18
Query 5: Route-wise Student Distribution.....	18
Query 6: Pickup Status Analysis.....	19
Query 7: Bus Capacity Analysis	19
14. USER INTERFACE DESIGN.....	20
14.1 Interface Features.....	20
14.2 Interface Screens	20
15. CRUD / LIVE DATABASE DEMONSTRATION	24
15.1 Step 1 — Initial State.....	24
15.2 Step 2 — Insertion	24
15.3 Step 3 — Database Verification (After Insert)	25
15.4 Step 4 — Deletion.....	25
15.5 Step 5 — Database Verification (After Delete).....	25
16. IMPLEMENTATION DETAILS	26
16.1 Final Database Statistics	26
16.2 Backend Implementation	26
16.3 Frontend Implementation.....	26
17. TESTING AND VALIDATION	27
18. CHALLENGES AND LIMITATIONS.....	28
18.1 Challenges Faced	28
18.2 Limitations	28
19. CONCLUSION, FUTURE SCOPE AND REFERENCES	28
19.1 Conclusion	28
19.2 Future Scope	29
19.3 References.....	29
A. GITHUB APPENDIX.....	30
Repository Structure	30

1. INTRODUCTION

Database Management Systems (DBMS) form the backbone of nearly every information system used in modern organizations, including educational institutions. A school transport operation is, in essence, a network of interrelated entities — students, drivers, buses, routes, and pickup and drop locations — whose relationships must be tracked accurately and consistently. As the number of students using transport services grows, so does the complexity of coordinating which student travels on which bus, along which route, picked up and dropped at which points, and whether the associated transport fee has been paid.

Manually maintained records, typically kept in registers or disconnected spreadsheets, struggle to represent these relationships reliably. A relational database, in contrast, is purpose-built to represent exactly this kind of structured, interconnected information: entities are stored in dedicated tables, and relationships between them are enforced through primary and foreign key constraints, ensuring that data remains consistent even as it grows in volume.

This project applies these principles to design and implement a Database Management System for a School Transport Route and Student Pickup Management System. The system is built around nine relational tables that together capture drivers, buses, routes, pickup points, drop points, students, transport assignments, daily pickup records, and transport fee records. A MySQL relational database serves as the data layer, while a web-based interface built with React, Vite, Node.js, and Express allows this data to be viewed and managed without direct interaction with SQL.

It is important to clarify that the routes, pickup points, and drop points used in this project are project-modelled transport routes that reference real Hyderabad locations for realism; they do not represent official Woxsen University shuttle routes. The motivation behind the project is academic — to apply database design, normalization, SQL querying, and full-stack integration concepts taught in the Database Management Systems course to a practical, relatable domain.

2. PROBLEM STATEMENT

School transport administration involves coordinating a large volume of interconnected information — student details, bus and driver assignments, route and stop information, daily pickup activity, and fee collection. When this information is maintained manually or without a structured database, several recurring problems arise:

- Managing large numbers of student transport records becomes error-prone as the student count grows.
- Duplicate or inconsistent records can appear when the same information is entered in multiple places.
- Tracking which student is linked to which route, pickup point, and drop point is difficult without an explicit relational structure.
- Assigning and updating bus and driver allocations manually is time-consuming and difficult to audit.
- Maintaining accurate pickup and drop information for every stop on every route is hard to keep current.
- Checking whether a student's transport fee has been paid, is pending, or is overdue requires manually cross-referencing separate records.
- Retrieving specific information — for example, all students assigned to a particular driver or route — is slow and unreliable without a query-based system.
- There is no user-friendly way for administrative staff to view or update transport information without technical database knowledge.

These problems motivate the need for a centralized, relational database system accessible through a simple web-based interface — the core objective of this project.

3. OBJECTIVES AND SCOPE

3.1 Objectives

1. Design a normalized relational database for school transport management.
2. Store and organize student transport information systematically.
3. Manage driver records, including licensing and experience details.
4. Manage bus records, including capacity and operational status.
5. Manage routes along with their associated pickup and drop points.
6. Maintain student transport assignments linking students to pickup and drop points.
7. Maintain daily pickup records for attendance and tracking purposes.
8. Maintain transport fee records, including due dates and payment status.
9. Implement relationships between entities using primary key and foreign key constraints.
10. Develop a web-based interface connected to the MySQL database using React, Vite, Node.js, and Express.
11. Support viewing, insertion, and deletion of student records through the interface.
12. Test database integrity and application functionality through representative queries and a live CRUD demonstration.

3.2 Scope

The system, as implemented, covers the following functional areas:

- Student management
- Driver management
- Bus management
- Route management
- Pickup point management
- Drop point management
- Student transport assignment management
- Pickup record tracking
- Transport fee tracking
- SQL-based querying and reporting
- Web-based administration with CRUD (Create, Read, Update, Delete) capability for core entities

The following enhancements are considered future scope and are not implemented in the current version of the project: GPS-based live tracking, automated parent/student notifications, and automated route optimization. These are discussed further in Section 19.

4. EXISTING SYSTEM AND PROPOSED SYSTEM

Many school transport operations continue to rely on manual or semi-manual methods for record keeping. This section compares the limitations of such an approach with the centralized database-driven approach proposed and implemented in this project.

Existing / Manual Approach	Proposed System
Records maintained manually in registers or spreadsheets	Centralized MySQL relational database
Searching for specific records is slow and manual	Structured relational tables enabling fast SQL-based search
Higher chance of duplicate or inconsistent entries	Primary key constraints prevent duplicate identifiers
Relationships between students, buses, routes, and drivers are tracked informally	Foreign-key relationships formally enforce data relationships
Limited centralized access to information	Web-based interface provides centralized, structured access
Reporting is manual and time-consuming	SQL-based analysis enables fast, accurate reporting
No systematic validation of data consistency	Referential integrity maintained through database constraints

5. SOFTWARE AND HARDWARE REQUIREMENTS

5.1 Software Requirements

The following software technologies were actually used in the design, implementation, and operation of the system:

Category	Technology Used
Database	MySQL
Frontend Framework	React (with Vite as the build tool)
Backend Runtime	Node.js
Backend Framework	Express.js
Database Driver	mysql2
Languages	JavaScript / JSX, HTML, CSS
Version Control	Git
Code Repository Hosting	GitHub
Development Environment	Visual Studio Code

5.2 Hardware Requirements

The following are reasonable minimum system requirements for development and operation of the application. These are standard academic guidelines rather than values measured through formal load testing.

Component	Minimum Requirement
Processor	Dual-core processor, 2.0 GHz or higher
RAM	4 GB (8 GB recommended)
Storage	At least 1 GB free disk space
Operating System	Windows 10 / 11, macOS, or Linux
Network	Internet connection for package installation and Git operations

6. SYSTEM ARCHITECTURE

The system follows a standard three-tier architecture, separating presentation, application logic, and data storage into distinct layers. This separation makes the system easier to develop, test, and maintain.

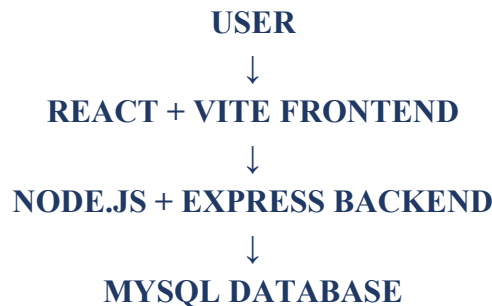


Figure 1: Three-Tier System Architecture

6.1 Frontend Layer (React + Vite)

The frontend provides the user interface through which administrative staff interact with the system. Its responsibilities include:

- Rendering the dashboard, data tables, and forms
- Providing search and filter controls for locating records
- Providing CRUD controls for adding and deleting records
- Displaying loading states, confirmation dialogs, and toast notifications

6.2 Backend Layer (Node.js + Express)

The backend exposes API endpoints consumed by the frontend. Its responsibilities include:

- Handling incoming HTTP requests from the frontend
- Validating request data before it reaches the database
- Executing database operations through the mysql2 driver
- Returning structured JSON responses to the frontend

6.3 Database Layer (MySQL)

The database layer stores all persistent data in nine relational tables, using primary and foreign key constraints to maintain consistency. Its responsibilities include:

- Storing structured data for drivers, buses, routes, pickup/drop points, students, assignments, pickup records, and fees
- Enforcing primary key uniqueness and foreign key referential integrity
- Executing SQL queries issued by the backend

7. DATABASE DESIGN

The database, named `school_transport_db`, consists of exactly nine relational tables. Each table represents a distinct entity in the school transport domain, and the tables are connected through primary and foreign key relationships described in detail in Section 9. The purpose of each table is summarized below.

Table	Purpose
<code>driver</code>	Stores details of each driver employed for school transport
<code>bus</code>	Stores details of each bus, including its assigned driver
<code>route</code>	Stores details of each transport route, including its assigned bus
<code>pickup_point</code>	Stores pickup stops associated with each route
<code>drop_point</code>	Stores drop stops associated with each route
<code>student</code>	Stores student details relevant to transport
<code>student_transport</code>	Links a student to a specific pickup point, drop point, and assignment period
<code>pickup_record</code>	Stores daily pickup activity records for each student
<code>transport_fee</code>	Stores transport fee amounts, due dates, and payment status for each student

8. ER DIAGRAM

The Entity Relationship (ER) diagram below represents the nine entities of the school transport database, along with their primary keys, foreign keys, and the cardinality of the relationships between them. The diagram reflects the final nine-table structure implemented in MySQL; no additional entities beyond these nine are part of the final database.

DRIVER PK: driver_id	$\rightarrow 1 : 1 \rightarrow$	BUS PK: bus_id FK: driver_id
		$\downarrow 1 : 1 \downarrow$
DROP_POINT PK: drop_id FK: route_id	$\leftarrow 1 : N \leftarrow$	ROUTE PK: route_id FK: bus_id
		$\downarrow 1 : N \downarrow$
		PICKUP_POINT PK: pickup_id FK: route_id
STUDENT PK: student_id	$\text{---} 1 : N \rightarrow$	STUDENT_TRANSPORT PK: assignment_id FK: student_id, pickup_id, drop_id
		(also linked from PICKUP_POINT and DROP_POINT, each 1 : N)
STUDENT PK: student_id	$\text{---} 1 : N \rightarrow$	PICKUP_RECORD PK: record_id FK: student_id
STUDENT PK: student_id	$\text{---} 1 : N \rightarrow$	TRANSPORT_FEE PK: fee_id FK: student_id

Figure 2: Entity Relationship Diagram of the School Transport Management System

9. RELATIONAL SCHEMA AND NORMALIZATION

9.1 Relational Schema Overview

The relational schema translates the ER diagram into nine concrete tables with explicit primary and foreign key constraints. The key relationships in the schema are as follows:

- driver → bus (1:1) — each bus is assigned exactly one driver for the given schedule
- bus → route (1:1) — each bus operates one route for the given schedule
- route → pickup_point (1:N) — a route has multiple pickup stops
- route → drop_point (1:N) — a route has multiple drop stops
- student → student_transport (1:N) — a student may have one or more transport assignment records over time
- pickup_point → student_transport (1:N) — a pickup point serves multiple students
- drop_point → student_transport (1:N) — a drop point serves multiple students
- student → pickup_record (1:N) — a student has multiple daily pickup records
- student → transport_fee (1:N) — a student may have multiple fee records across billing periods

Referential integrity is enforced throughout: a foreign key value in a child table (for example, driver_id in bus) must correspond to an existing primary key value in the parent table (driver). This prevents orphaned or inconsistent references and ensures that operations such as deleting a student correctly affect its dependent records.

9.2 Normalization

The schema was designed with normalization principles in mind to minimise data redundancy and avoid update anomalies.

First Normal Form (1NF): Every table stores atomic, single-valued attributes in each column, with no repeating groups. For example, a student's transport assignment is not stored as a combined field containing pickup point, drop point, and dates together; instead, each of these is a distinct attribute in the student_transport table.

Second Normal Form (2NF): Every table uses a single-column primary key (for example, student_id, driver_id, assignment_id), so partial dependency on part of a composite key does not arise. All non-key attributes depend on the whole of the primary key.

Third Normal Form (3NF): Non-key attributes depend only on the primary key and not on other non-key attributes. For instance, driver_name and license_no are stored only in the driver table rather than being duplicated in the bus table; the bus table instead stores a driver_id foreign key reference. This avoids redundant storage of driver information across multiple buses.

This normalization rationale reflects the design intent followed while structuring the nine tables; it is presented as the basis for the schema design rather than the output of a separate formal normalization exercise performed after the fact.

10. DATA DICTIONARY

The data dictionary below documents every attribute of the nine tables in the database, including data type, key/constraint information, and a short description. A note on naming: the student table's class column is displayed to end users in the web interface as "Course"; the underlying database column itself remains named class and has not been renamed at the database level.

driver

Attribute	Data Type	Key / Constraint	Description
driver_id	INT	Primary Key	Unique identifier for each driver
driver_name	VARCHAR(100)	NOT NULL	Full name of the driver
phone	VARCHAR(15)	NOT NULL	Contact number of the driver
license_no	VARCHAR(20)	UNIQUE, NOT NULL	Driving license number
experience_years	INT	NOT NULL	Years of driving experience
status	VARCHAR(20)	NOT NULL	Current employment status (e.g., Active)

bus

Attribute	Data Type	Key / Constraint	Description
bus_id	INT	Primary Key	Unique identifier for each bus
bus_number	VARCHAR(20)	UNIQUE, NOT NULL	Vehicle registration number
capacity	INT	NOT NULL	Total seating capacity of the bus
bus_model	VARCHAR(50)	—	Model or make of the bus
status	VARCHAR(20)	—	Operational status of the bus
driver_id	INT	Foreign Key → driver(driver_id)	Driver assigned to the bus

route

Attribute	Data Type	Key / Constraint	Description
route_id	INT	Primary Key	Unique identifier for each route
route_name	VARCHAR(50)	NOT NULL	Name of the route (e.g., Route A)
start_location	VARCHAR(100)	NOT NULL	Starting location of the route
end_location	VARCHAR(100)	NOT NULL	Ending location of the route
distance_km	DECIMAL(5,2)	—	Total route distance in kilometres
bus_id	INT	Foreign Key → bus(bus_id)	Bus assigned to operate the route

pickup_point

Attribute	Data Type	Key / Constraint	Description
pickup_id	INT	Primary Key	Unique identifier for each pickup point
pickup_name	VARCHAR(100)	NOT NULL	Name of the pickup stop
location	VARCHAR(150)	—	Descriptive location of the stop
pickup_time	TIME	—	Scheduled pickup time at the stop
route_id	INT	Foreign Key → route(route_id)	Route associated with the pickup point

Attribute	Data Type	Key / Constraint	Description
stop_order	INT	—	Sequence of the stop along the route

drop_point

Attribute	Data Type	Key / Constraint	Description
drop_id	INT	Primary Key	Unique identifier for each drop point
drop_name	VARCHAR(100)	NOT NULL	Name of the drop stop
location	VARCHAR(150)	—	Descriptive location of the stop
drop_time	TIME	—	Scheduled drop time at the stop
stop_order	INT	—	Sequence of the stop along the route
route_id	INT	Foreign Key → route(route_id)	Route associated with the drop point

student

Attribute	Data Type	Key / Constraint	Description
student_id	VARCHAR(10)	Primary Key	Unique identifier for each student (e.g., S001)
student_name	VARCHAR(100)	NOT NULL	Full name of the student
class	VARCHAR(20)	NOT NULL	Academic class/programme of the student
section	VARCHAR(10)	—	Section within the class
parent_name	VARCHAR(100)	—	Name of the parent or guardian
parent_contact	VARCHAR(15)	—	Contact number of the parent or guardian

student_transport

Attribute	Data Type	Key / Constraint	Description
assignment_id	INT	Primary Key	Unique identifier for each transport assignment
student_id	VARCHAR(10)	Foreign Key → student(student_id)	Student linked to the assignment
pickup_id	INT	Foreign Key → pickup_point(pickup_id)	Assigned pickup point
drop_id	INT	Foreign Key → drop_point(drop_id)	Assigned drop point
start_date	DATE	—	Date the assignment begins
end_date	DATE	—	Date the assignment ends
status	VARCHAR(20)	—	Current status of the assignment (e.g., Active)

pickup_record

Attribute	Data Type	Key / Constraint	Description
record_id	INT	Primary Key	Unique identifier for each pickup record
student_id	VARCHAR(10)	Foreign Key → student(student_id)	Student associated with the record
pickup_date	DATE	NOT NULL	Date of the pickup
pickup_time	TIME	—	Actual recorded pickup time
status	VARCHAR(20)	—	Status of the pickup (e.g., Picked Up, Missed)
remarks	VARCHAR(255)	—	Additional notes about the pickup

transport_fee

Attribute	Data Type	Key / Constraint	Description
fee_id	INT	Primary Key	Unique identifier for each fee record
student_id	VARCHAR(10)	Foreign Key → student(student_id)	Student associated with the fee
amount	DECIMAL(10,2)	NOT NULL	Transport fee amount due
due_date	DATE	—	Date the fee is due
payment_date	DATE	—	Date the fee was paid (if applicable)
payment_status	VARCHAR(20)	—	Payment status (e.g., Paid, Pending)

11. DATABASE IMPLEMENTATION

The relational schema described in Sections 7–10 was implemented as a live MySQL database named `school_transport_db`. This section presents representative Data Definition Language (DDL) statements used to create the database and two of its tables; the remaining seven tables follow the same pattern, with primary keys, foreign keys, and constraints applied according to the data dictionary in Section 10.

11.1 Database Creation

```
CREATE DATABASE school_transport_db;
USE school_transport_db;
```

11.2 Representative Table Creation

```
CREATE TABLE driver (
  driver_id INT PRIMARY KEY,
  driver_name VARCHAR(100) NOT NULL,
  phone VARCHAR(15) NOT NULL,
  license_no VARCHAR(20) UNIQUE NOT NULL,
  experience_years INT NOT NULL,
  status VARCHAR(20) NOT NULL
);

CREATE TABLE bus (
  bus_id INT PRIMARY KEY,
  bus_number VARCHAR(20) UNIQUE NOT NULL,
  capacity INT NOT NULL,
  bus_model VARCHAR(50),
  status VARCHAR(20),
  driver_id INT,
  FOREIGN KEY (driver_id) REFERENCES driver(driver_id)
);
```

11.3 Representative Alteration

The `ALTER TABLE` statement was used during development to add or adjust constraints after a table had already been created, for example:

```
ALTER TABLE student_transport
  ADD CONSTRAINT fk_drop_point
  FOREIGN KEY (drop_id) REFERENCES drop_point(drop_id);
```

These representative statements illustrate the constraint patterns — `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, and `ALTER TABLE` — applied consistently across all nine tables, rather than reproducing the complete schema script in full.

12. DDL AND DML OPERATIONS

In addition to the DDL statements shown in Section 11, standard Data Manipulation Language (DML) operations — INSERT, UPDATE, DELETE, and SELECT — were used throughout development and operation of the system. Representative examples are shown below.

12.1 INSERT

Adds a new record to a table. For example, inserting a new student record:

```
INSERT INTO student
(student_id, student_name, class, section, parent_name, parent_contact)
VALUES
('S001', 'Aarav Sharma', 'AIML', 'A', 'Mr. Sharma', '9876543210');
```

12.2 UPDATE

Modifies existing records. For example, marking a transport fee as paid:

```
UPDATE transport_fee
SET payment_status = 'Paid', payment_date = CURDATE()
WHERE fee_id = 101;
```

12.3 DELETE

Removes a record from a table. For example, removing a student record:

```
DELETE FROM student
WHERE student_id = 'S286';
```

12.4 SELECT

Retrieves records from a table. For example, retrieving all students in a given class:

```
SELECT *
FROM student
WHERE class = 'AIML';
```

13. SQL QUERIES AND RESULTS

This section presents seven representative SQL queries executed against the populated database, each combined with multiple JOINS or aggregate functions to demonstrate the relationships established between tables. Actual query output has not been reproduced here as a screenshot was not available at the time of writing; each query is accompanied by a clearly labelled placeholder for the corresponding result screenshot.

Query 1: Students Under a Specific Driver

Retrieves the complete transport chain for all students assigned to buses driven by a specific driver (Mahesh Singh), joining student, student_transport, pickup_point, drop_point, route, bus, and driver.

```
SELECT
s.student_id,
s.student_name,
pp.pickup_name,
dp.drop_name,
r.route_name,
b.bus_number,
d.driver_name
FROM student s
```

```
JOIN student_transport st ON s.student_id = st.student_id
JOIN pickup_point pp ON st.pickup_id = pp.pickup_id
JOIN drop_point dp ON st.drop_id = dp.drop_id
JOIN route r ON pp.route_id = r.route_id
JOIN bus b ON r.bus_id = b.bus_id
JOIN driver d ON b.driver_id = d.driver_id
WHERE d.driver_name = 'Mahesh Singh';
```

Query 2: Route-wise Students

Lists every student along with their pickup point, grouped implicitly by route, to show which students travel on each route.

```
SELECT
    r.route_name,
    s.student_name,
    pp.pickup_name
FROM student s
JOIN student_transport st ON s.student_id = st.student_id
JOIN pickup_point pp ON st.pickup_id = pp.pickup_id
JOIN route r ON pp.route_id = r.route_id
ORDER BY r.route_name;
```

Query 3: Complete Transport Details

Produces the full transport detail for every student by joining all relevant tables: student, assignment, pickup point, drop point, route, bus, and driver.

```
SELECT
    s.student_id, s.student_name,
    pp.pickup_name, dp.drop_name,
    r.route_name, b.bus_number,
    d.driver_name, st.status
FROM student s
JOIN student_transport st ON s.student_id = st.student_id
JOIN pickup_point pp ON st.pickup_id = pp.pickup_id
JOIN drop_point dp ON st.drop_id = dp.drop_id
JOIN route r ON pp.route_id = r.route_id
JOIN bus b ON r.bus_id = b.bus_id
JOIN driver d ON b.driver_id = d.driver_id;
```

Query 4: Pending Transport Fees

Identifies all students whose transport fee payment status is currently pending.

```
SELECT
    s.student_id,
    s.student_name,
    tf.amount,
    tf.due_date,
    tf.payment_status
FROM student s
JOIN transport_fee tf ON s.student_id = tf.student_id
WHERE tf.payment_status = 'Pending';
```

Query 5: Route-wise Student Distribution

Counts the number of students assigned to each route, using GROUP BY to aggregate across pickup points belonging to the same route.

```
SELECT
    r.route_name,
    COUNT(st.assignment_id) AS total_students
FROM route r
JOIN pickup_point pp ON pp.route_id = r.route_id
JOIN student_transport st ON st.pickup_id = pp.pickup_id
GROUP BY r.route_name
ORDER BY total_students DESC;
```

Query 6: Pickup Status Analysis

Aggregates pickup_record entries by status to analyse how many pickups were completed versus missed.

```
SELECT
    status,
    COUNT(*) AS total_records
FROM pickup_record
GROUP BY status;
```

Query 7: Bus Capacity Analysis

Compares each bus's seating capacity against the number of students actually assigned to it, to analyse capacity utilisation.

```
SELECT
    b.bus_number,
    b.capacity,
    COUNT(st.assignment_id) AS assigned_students
FROM bus b
JOIN route r ON r.bus_id = b.bus_id
JOIN pickup_point pp ON pp.route_id = r.route_id
JOIN student_transport st ON st.pickup_id = pp.pickup_id
GROUP BY b.bus_number, b.capacity;
```

14. USER INTERFACE DESIGN

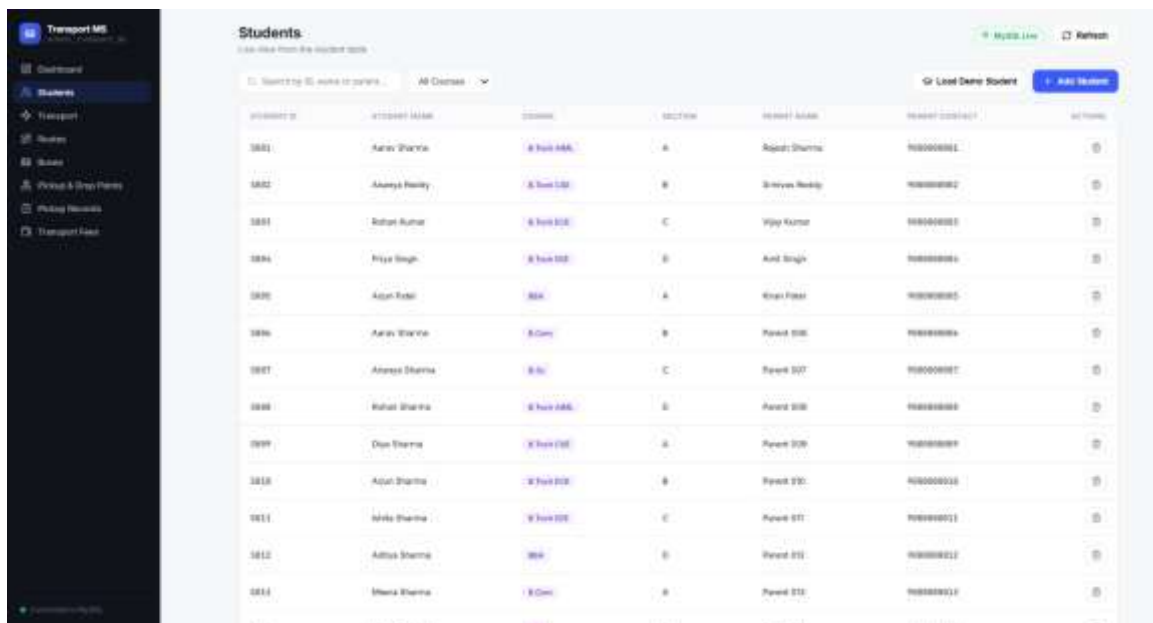
The user interface was developed using React with Vite as the build tool, providing a responsive, single-page administrative application for managing the school transport database. The interface communicates with the Node.js/Express backend through REST API calls, which in turn query the MySQL database.

14.1 Interface Features

- Dashboard displaying summary statistics across all entities
- Dedicated management screens for Students, Transport, Routes, Buses, Pickup/Drop Points, Pickup Records, and Transport Fees
- Search functionality for quickly locating specific records
- Filtering controls for narrowing displayed records by criteria such as route or status
- Pagination for navigating large record sets such as the 285-student list
- Add Student functionality for inserting new records
- Delete Student functionality for removing records
- Loading states shown while data is being fetched from the backend
- Confirmation dialogs shown before destructive actions such as deletion
- Toast notifications confirming the success or failure of an action
- A live MySQL connection indicator showing the current database connectivity status

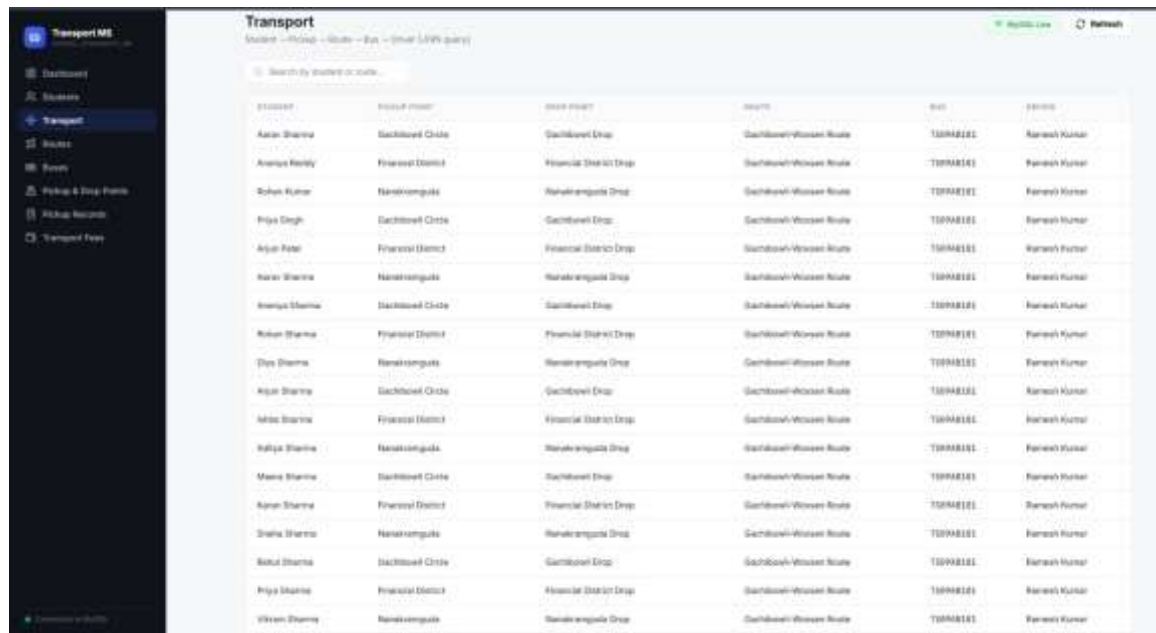
14.2 Interface Screens

Figure 3: Main Dashboard



STUDENT ID	STUDENT NAME	CONTACT	SECTION	PARENT NAME	PARENT CONTACT	ACTIONS
0801	Arun Sharma	8765 4321	A	Rajesh Sharma	9876 5432	[Edit] [Delete]
0802	Ananya Reddy	8765 4321	B	Shravan Reddy	9876 5432	[Edit] [Delete]
0803	Robert Kumar	8765 4321	C	Vijay Kumar	9876 5432	[Edit] [Delete]
0804	Priya Singh	8765 4321	D	Anil Singh	9876 5432	[Edit] [Delete]
0805	Arun Patel	8765	A	Kiran Patel	9876 5432	[Edit] [Delete]
0806	Arun Sharma	8765	B	Parth 006	9876 5432	[Edit] [Delete]
0807	Ananya Sharma	8765	C	Parth 007	9876 5432	[Edit] [Delete]
0808	Robert Sharma	8765 4321	D	Parth 008	9876 5432	[Edit] [Delete]
0809	Dan Sharma	8765 4321	A	Parth 009	9876 5432	[Edit] [Delete]
0810	Arun Sharma	8765 4321	B	Parth 010	9876 5432	[Edit] [Delete]
0811	Arun Sharma	8765 4321	C	Parth 011	9876 5432	[Edit] [Delete]
0812	Arun Sharma	8765	D	Parth 012	9876 5432	[Edit] [Delete]
0813	Mona Sharma	8765	B	Parth 013	9876 5432	[Edit] [Delete]

Figure 4: Student Management Screen

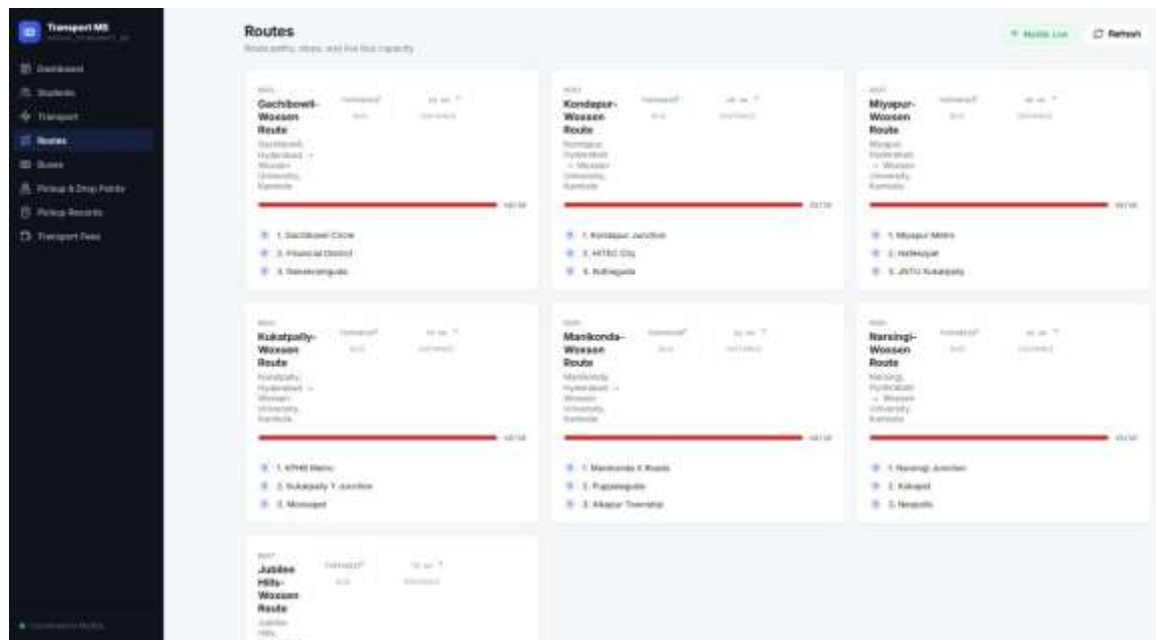


Transport
Routes -> Routes -> Routes -> Routes (with query)

Search by student or route

Student	From	To	Route	Route ID	Route
Aashu Sharma	Gachibowli Circle	Gachibowli Drop	Gachibowli-Woxsen Route	10000001	Kamath Kumar
Ananya Reddy	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000002	Kamath Kumar
Arjun Kumar	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000003	Kamath Kumar
Arjun Singh	Gachibowli Circle	Gachibowli Drop	Gachibowli-Woxsen Route	10000004	Kamath Kumar
Arjun Patel	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000005	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000006	Kamath Kumar
Ashu Sharma	Gachibowli Circle	Gachibowli Drop	Gachibowli-Woxsen Route	10000007	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000008	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000009	Kamath Kumar
Ashu Sharma	Gachibowli Circle	Gachibowli Drop	Gachibowli-Woxsen Route	10000010	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000011	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000012	Kamath Kumar
Ashu Sharma	Gachibowli Circle	Gachibowli Drop	Gachibowli-Woxsen Route	10000013	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000014	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000015	Kamath Kumar
Ashu Sharma	Gachibowli Circle	Gachibowli Drop	Gachibowli-Woxsen Route	10000016	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000017	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000018	Kamath Kumar
Ashu Sharma	Gachibowli Circle	Gachibowli Drop	Gachibowli-Woxsen Route	10000019	Kamath Kumar
Ashu Sharma	Financial District	Financial District Drop	Gachibowli-Woxsen Route	10000020	Kamath Kumar

Figure 5: Transport Management Screen



Routes
Route info, status, and live bus capacity

Routes Link Refresh

Route	From	To	Route	From	To	Route	From	To
Gachibowli-Woxsen Route	Gachibowli Circle	Gachibowli Drop	Kondapur-Woxsen Route	Kondapur Circle	Kondapur Drop	Mysore-Woxsen Route	Mysore Circle	Mysore Drop
Kukatpally-Woxsen Route	Kukatpally Circle	Kukatpally Drop	Manikonda-Woxsen Route	Manikonda Circle	Manikonda Drop	Narasingi-Woxsen Route	Narasingi Circle	Narasingi Drop
Jubilee Hills-Woxsen Route	Jubilee Hills Circle	Jubilee Hills Drop						

Figure 6: Route Management Screen

Transport M8

Dashboard

Students

Transport

Routes

Routes

Participating Drop Points

Participating Routes

Transport Fees

Transport Fees

Fee methods and payment status

Search by student or status

ID	Student	Amount	Due Date	Payment Date	Status
F001	Rohan Sharma	₹1,500	30/09/2024	01/10/2024	Paid
F002	Aanya Reddy	₹1,500	30/09/2024	01/10/2024	Paid
F003	Ashika Reddy	₹1,500	30/09/2024	—	Pending
F004	Shiva Singh	₹1,500	30/09/2024	06/10/2024	Paid
F005	Ajaya Patel	₹1,500	30/09/2024	—	Pending
F006	Rohan Sharma	₹30,000	30/09/2024	01/10/2024	Paid
F007	Aanya Sharma	₹11,000	30/09/2024	01/10/2024	Paid
F008	Rohan Sharma	₹25,000	30/09/2024	06/10/2024	Paid
F009	Shiva Sharma	₹24,000	30/09/2024	03/10/2024	Loan
F010	Ajaya Sharma	₹30,000	30/09/2024	—	Pending
F011	Ashika Sharma	₹21,000	30/09/2024	01/10/2024	Paid
F012	Ashika Sharma	₹24,000	30/09/2024	07/10/2024	Paid
F013	Mohan Sharma	₹20,000	30/09/2024	06/10/2024	Paid
F014	Karan Sharma	₹30,000	30/09/2024	01/10/2024	Loan
F015	Shruti Sharma	₹17,000	30/09/2024	—	Pending

Page 1 of 10 | 100 rows selected

Figure 10: Transport Fee Screen

15. CRUD / LIVE DATABASE DEMONSTRATION

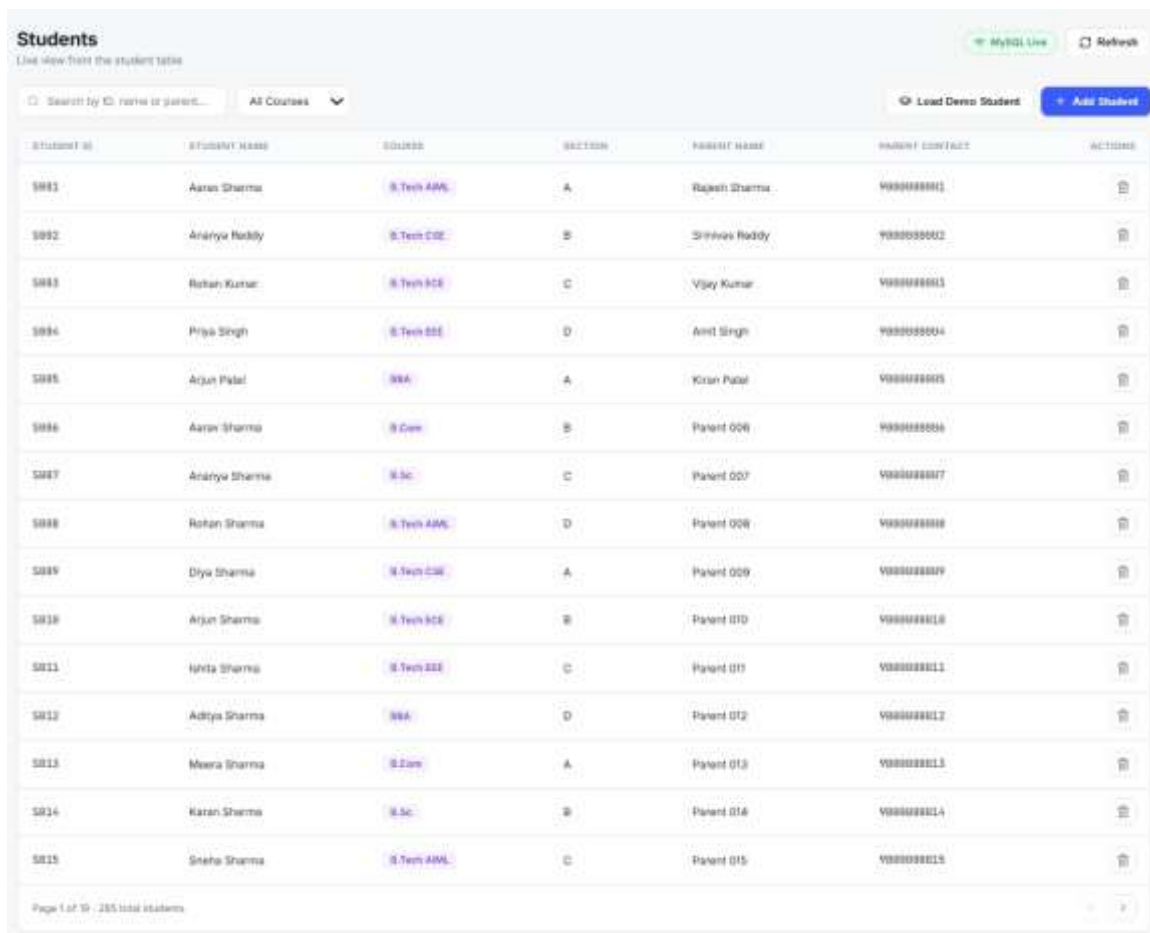
A live demonstration of Create, Read, and Delete operations was carried out during Presentation III to verify that the web interface correctly reflects changes in the underlying MySQL database. The demonstration used a sample student record with the following details:

Field	Value
Student ID	S286
Student Name	Trilochan Demo
Course	B.Tech AIML
Section	A
Parent Name	Demo Parent
Parent Contact	9876543210

The demonstration followed a four-step sequence: VIEW → INSERT → DATABASE VERIFICATION → DELETE → DATABASE VERIFICATION.

15.1 Step 1 — Initial State

Before insertion, the student table contained 285 records.



Student ID	Student Name	Course	Section	Parent Name	Parent Contact	Actions
S001	Aarav Sharma	B.Tech AIML	A	Rajesh Sharma	9880000001	
S002	Ananya Reddy	B.Tech CSE	B	Srinivas Reddy	9880000002	
S003	Rohan Kumar	B.Tech ICS	C	Vijay Kumar	9880000003	
S004	Priya Singh	B.Tech EEE	D	Amit Singh	9880000004	
S005	Arjun Patel	BBA	A	Kiran Patel	9880000005	
S006	Aarav Sharma	B.Com	B	Parent 006	9880000006	
S007	Ananya Sharma	B.Sc	C	Parent 007	9880000007	
S008	Rohan Sharma	B.Tech AIML	D	Parent 008	9880000008	
S009	Divya Sharma	B.Tech CSE	A	Parent 009	9880000009	
S010	Arjun Sharma	B.Tech ICS	B	Parent 010	9880000010	
S011	Ishita Sharma	B.Tech EEE	C	Parent 011	9880000011	
S012	Aditya Sharma	BBA	D	Parent 012	9880000012	
S013	Meera Sharma	B.Com	A	Parent 013	9880000013	
S014	Karan Sharma	B.Sc	B	Parent 014	9880000014	
S015	Sneha Sharma	B.Tech AIML	C	Parent 015	9880000015	

Figure 11: Student List Before Insertion

15.2 Step 2 — Insertion

The sample student S286 was added through the Add Student form in the web interface.

The screenshot shows a web interface titled 'Students'. It includes a search bar, a dropdown for 'All Courses', and buttons for 'Load Demo Student' and 'Add Student'. Below these is a table with columns: STUDENT ID, STUDENT NAME, COURSE, SECTION, PARENT NAME, PARENT CONTACT, and ACTIONS. The table contains one row for student ID 'S286', named 'Trilochan Demo', in course 'B.Tech AIML', section 'A', with parent 'Demo Parent' and contact '9876543210'. A pagination bar at the bottom indicates 'Page 1 of 1 | 1 total students'.

STUDENT ID	STUDENT NAME	COURSE	SECTION	PARENT NAME	PARENT CONTACT	ACTIONS
S286	Trilochan Demo	B.Tech AIML	A	Demo Parent	9876543210	[Icon]

Figure 12: Student List After Insertion (286 records)

15.3 Step 3 — Database Verification (After Insert)

The insertion was verified directly in MySQL using the following query:

```
SELECT *
FROM student
WHERE student_id = 'S286';
```

```
mysql> SELECT *
-> FROM student
-> WHERE student_id = 'S286';
```

student_id	student_name	class	section	parent_name	parent_contact
S286	Trilochan Demo	B.Tech AIML	A	Demo Parent	9876543210

1 row in set (0.00 sec)

Figure 13: MySQL Verification After Insertion

15.4 Step 4 — Deletion

The sample record was then deleted through the web interface, with a confirmation dialog shown before the action was executed.

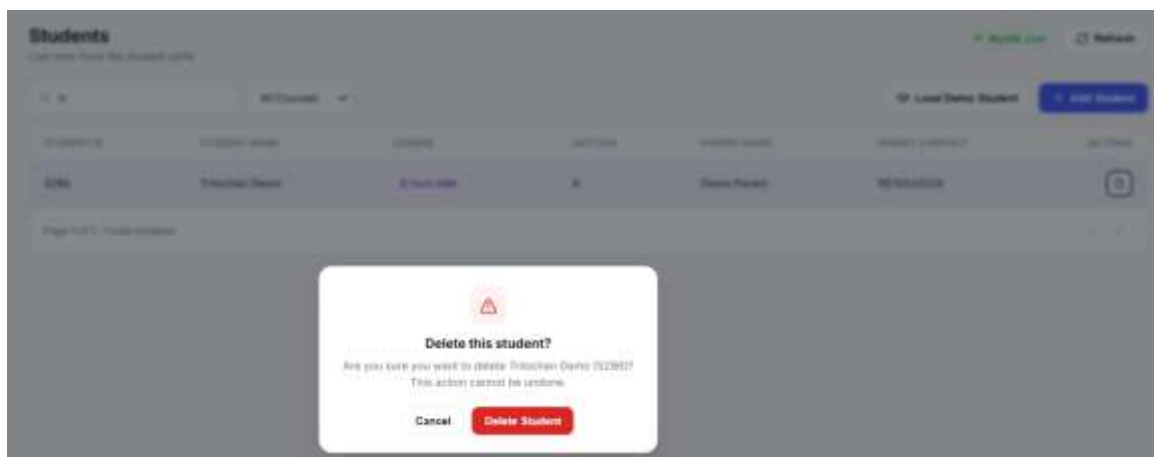


Figure 14: Delete Confirmation Dialog

15.5 Step 5 — Database Verification (After Delete)

The deletion was verified in MySQL; querying for S286 returned an Empty Set, and the total student count returned to its original value.

```
SELECT *
FROM student
```

```
WHERE student_id = 'S286';
-- Expected: Empty Set

SELECT COUNT(*) AS total_students
FROM student;
-- Expected: 285
```

```
mysql> SELECT *
      -> FROM student
      -> WHERE student_id = 'S286';
Empty set (0.00 sec)
```

Figure 17: MySQL Empty Set Verification After Deletion

This demonstration confirmed that the web interface correctly performs INSERT and DELETE operations against the live MySQL database, and that the student count returns to its correct baseline of 285 after the demonstration record is removed.

16. IMPLEMENTATION DETAILS

This section summarises the finalized dataset populated into the database and the overall implementation statistics.

16.1 Final Database Statistics

Table	Record Count
Driver	7
Bus	7
Route	7
Pickup Point	21
Drop Point	21
Student	285
Student Transport	285
Pickup Record	855
Transport Fee	285
TOTAL	1,773

Note that total bus capacity across all 7 buses equals 285, and the populated dataset assigns all 285 student seats across the fleet, meaning the transport system operates at full assigned capacity under the current dataset.

16.2 Backend Implementation

The backend was implemented using Node.js and the Express.js framework, exposing REST endpoints for each of the nine entities. The mysql2 driver was used to execute parameterised SQL queries against the MySQL database, reducing the risk of SQL injection and keeping query logic centralised in the backend layer.

16.3 Frontend Implementation

The frontend was implemented using React, bundled with Vite for fast development and build performance. Each entity (students, drivers, buses, routes, pickup/drop points, pickup records, and transport fees) has a corresponding screen that fetches data from the backend API and renders it in a searchable, filterable, paginated table.

17. TESTING AND VALIDATION

Functional testing was carried out across the application to verify that the interface and database behave correctly, and that database integrity checks hold under normal operations.

Test Case	Expected Result	Actual Result	Status
Dashboard loading	Summary statistics load correctly from the backend	Statistics loaded as expected	Pass
Student viewing	Full list of 285 students displays with pagination	List displayed correctly	Pass
Student insertion	New student (S286) added and visible immediately	Student added and visible	Pass
Duplicate student ID handling	Database rejects a duplicate primary key	Insertion rejected by primary key constraint	Pass
Student deletion	Student S286 removed and no longer visible	Student removed successfully	Pass
Foreign-key / dependent-record protection	Deletion respects dependent records in related tables	Dependent records handled as expected	Pass
Route viewing	All 7 routes display correctly	Routes displayed correctly	Pass
Bus viewing	All 7 buses display with correct capacity and driver	Buses displayed correctly	Pass
Pickup/drop viewing	21 pickup points and 21 drop points display correctly	Points displayed correctly	Pass
Pickup records	855 pickup records accessible and filterable	Records accessible as expected	Pass
Transport fees	285 fee records display with correct payment status	Fee records displayed correctly	Pass
Database connectivity	Live MySQL connection indicator reflects actual status	Indicator reflected live status correctly	Pass
SQL queries	All seven representative queries execute without error	Queries executed successfully	Pass

These tests, together with database-level integrity checks on primary and foreign key constraints, confirm that the core CRUD functionality and relational structure of the system behave as designed.

18. CHALLENGES AND LIMITATIONS

18.1 Challenges Faced

- Designing consistent relationships across multiple interconnected transport entities (driver, bus, route, pickup/drop points, student).
- Maintaining foreign-key consistency while populating a large dataset of 1,773 records across nine tables.
- Handling dependent records correctly when a parent record (such as a student) is deleted.
- Connecting the React frontend and Node.js/Express backend reliably to the MySQL database.
- Managing and querying a dataset of this size efficiently during development and testing.
- Testing UI changes consistently against the live database to ensure displayed data remained accurate.
- Maintaining a clean, usable interface while surfacing the full complexity of nine related tables.

18.2 Limitations

- No GPS-based live bus tracking is currently implemented.
- No automated parent or student notification system is currently implemented.
- No automated route optimisation engine is currently implemented.
- The routes, pickup points, and drop points are project-modelled using Hyderabad reference locations and do not represent official Woxsen University shuttle routes.
- Role-based authentication and access control are not implemented as a primary feature of the current version.

19. CONCLUSION, FUTURE SCOPE AND REFERENCES

19.1 Conclusion

This project set out to address the difficulty of managing school transport information using manual, disconnected record-keeping methods. By designing a normalized relational schema of nine interconnected tables and implementing it in MySQL, the project demonstrates how core Database Management Systems concepts — entity-relationship modelling, normalization, primary and foreign key constraints, and SQL querying — can be applied to build a coherent, centralized data model for a real-world domain.

The accompanying web-based interface, built with React, Vite, Node.js, and Express, demonstrates practical full-stack integration with a relational database, allowing administrative users to view, search, insert, and delete records without direct SQL interaction. The live CRUD demonstration and the set of seven analytical SQL queries validate that the schema's relationships function correctly and that the system can support meaningful reporting, such as route-wise student distribution and pending fee analysis. Overall, the project successfully

demonstrates relational database design, SQL implementation, referential data integrity, and full-stack CRUD interaction as a practical application of DBMS coursework.

19.2 Future Scope

The following enhancements are identified as potential future work and are not implemented in the current version of the project:

- GPS-based live bus tracking
- Parent and student notification system
- Automated transport fee payment reminders
- Route optimisation engine
- Driver-facing mobile application
- Attendance integration with pickup records
- Analytics dashboard with richer visualisations
- Role-based authentication and access control
- Automated alerts for route delays
- Cloud deployment of the full application stack

19.3 References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts. McGraw-Hill Education.
2. Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems. Pearson Education.
3. MySQL 8.0 Reference Manual — Oracle Corporation. <https://dev.mysql.com/doc/>
4. React Documentation. <https://react.dev>
5. Node.js Documentation. <https://nodejs.org/docs>
6. Express.js Documentation. <https://expressjs.com>
7. Vite Documentation. <https://vitejs.dev>

A. GITHUB APPENDIX

The complete source code and project artefacts are maintained in the following GitHub repository:

Repository Name: DBMS-Course-Project

Repository Structure

```
dbms-transport-ui/  
├── backend/  
├── frontend/  
├── Presentation-I/  
├── Presentation-II/  
├── Presentation-III/  
├── Project-Report/  
├── .gitignore  
└── README.md
```

The repository contains the backend source code (Node.js/Express and MySQL integration), the frontend source code (React/Vite), the materials for Presentation I, Presentation II, and Presentation III, and this project report.